SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

Code of Conduct:

I K. Lahari (192325118) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: K. Lahari

1: **HTTP Response Analysis**

A web server returns the following HTTP response header:
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with Content-Length and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

Answer:

Code:

```
<!DOCTYPE html>

<html>

<head>

   <title>HTTP Response Analysis</title>

</head>

<body>

   <h2>HTTP Response Header Validation</h2>

   <pre id="result"></pre>

   <script>

      let status = 200;

      let contentLength = -150;

      let date = new Date("2026-04-01T10:00:00Z");

      let output = "Status: " + status + " OK\n";



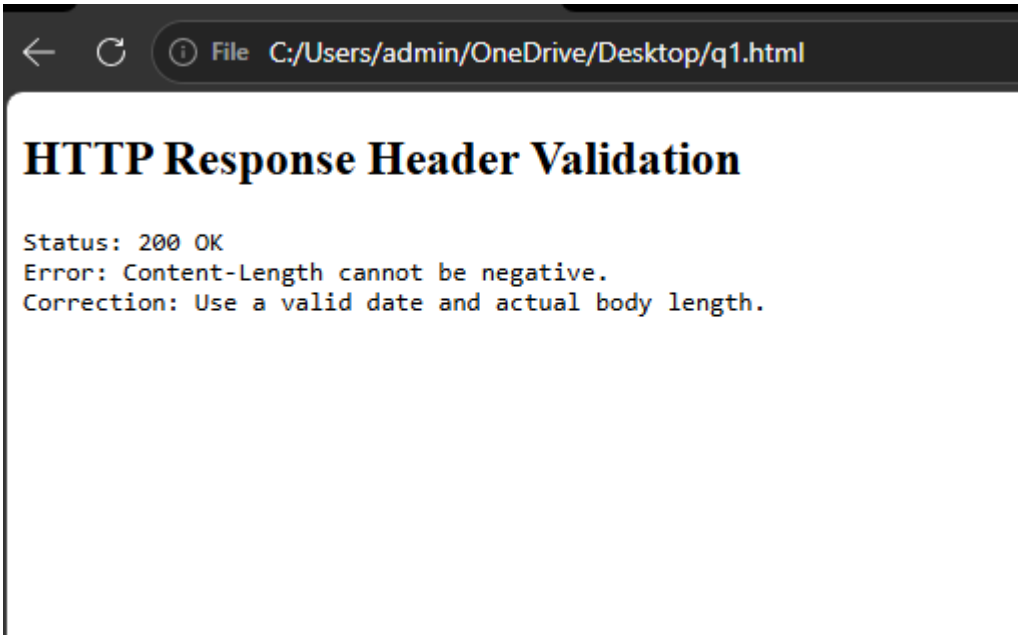
      if (contentLength < 0) {

         output +=
```

```
            "Error: Content-Length cannot be negative.\n";
        }
        if (date.getUTCDay() !== 3) {
            output +=
                "Error: Invalid weekday in Date header.\n";
        }
        output +=
            "Correction: Use a valid date and actual body length.";

        document.getElementById("result").textContent = output;
    </script>
</body>
</html>
```

Output:



1. Identify incorrect or invalid fields.

The Content-Length field contains a negative value, which is invalid. The Date field also contains an incorrect weekday because April 1, 2026, is Wednesday, not Monday.

2. Analyze the Content-Length issue.

Content-Length specifies the response body size in bytes. A negative value is invalid and may cause the response to be rejected or processed incorrectly.

3. Does the status code match the response correctness?

The status code 200 OK indicates successful processing of the request. However, it does not guarantee that every response header is valid.

4. Suggest corrections.

Use the correct weekday and a non-negative Content-Length that matches the actual response body size.

2: **DOM Structure Analysis**

Consider the following DOM tree representation:

```
<html>
 <head>
 <title>TestPage</title>
</head>
 <body>
 <div>
 <p>HelloWorld
 </div>
 </body>
 </html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

Answer:

Code:

```
<!DOCTYPE html>
<html>
<head>
```

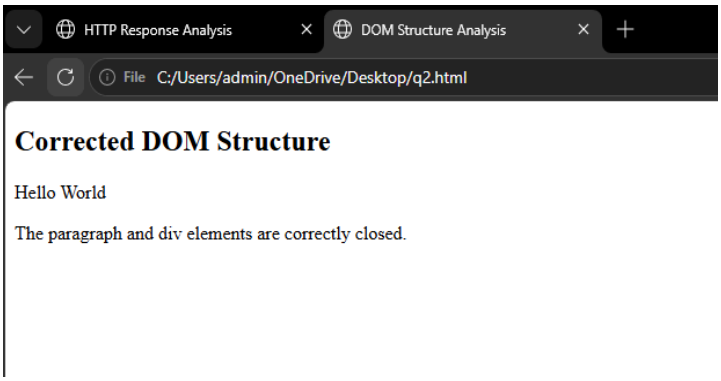
```
    <title>DOM Structure Analysis</title>
</head>
<body>
    <h2>Corrected DOM Structure</h2>

    <div>
        <p>Hello World</p>
    </div>

    <p id="result"></p>

    <script>
        document.getElementById("result").textContent =
            "The paragraph and div elements are correctly closed.";
    </script>
</body>
</html>
```

Output:



1. Identify missing or improperly closed elements.

The original code is missing the closing </p> tag before the closing tag.

2. How will the browser interpret the structure?

The browser attempts to correct malformed HTML and may automatically close the paragraph when it encounters the closing div tag.

3. What is the impact on rendering and layout?

Incorrect nesting may cause unexpected spacing, layout differences, or an unintended DOM structure.

4. Suggest corrections.

Add the closing </p> tag before to maintain proper HTML structure.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

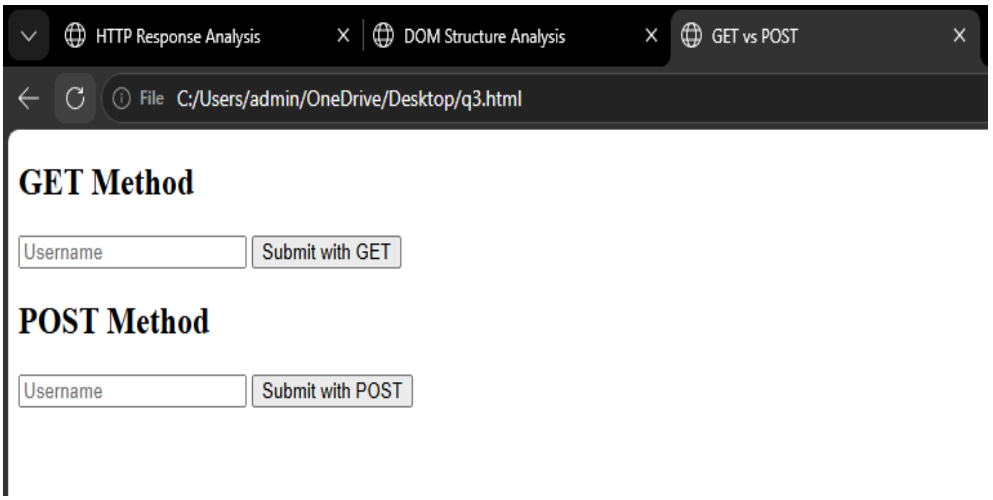
Answer:

Code:

```
<!DOCTYPE html>
<html>
<head>
  <title>GET vs POST</title>
</head>
<body>
  <h2>GET Method</h2>
  <form action="" method="GET">
    <input type="text" name="username"
        placeholder="Username" required>
    <button type="submit">Submit with GET</button>
  </form>

  <h2>POST Method</h2>
  <form action="" method="POST">
    <input type="text" name="username"
        placeholder="Username" required>
    <button type="submit">Submit with POST</button>
  </form>
</body>
</html>
```

Output:



1. Interpret the differences between GET and POST.

GET sends form data through URL query parameters, whereas POST sends form data in the request body.

2. Which method is more suitable for login?

POST is more suitable because login credentials are not placed directly in the URL.

3. Identify security risks of GET.

Credentials may be exposed through browser history, server logs, shared URLs, and referrer information.

4. Suggest secure login practices.

- Use POST for submitting login credentials.
- Use HTTPS to encrypt communication.
- Store passwords using secure password hashing.
- Use secure session cookies.
- Apply rate limiting to prevent repeated login attempts.
-

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
```

<p>This is a paragraph
<img src="image.jpg">
</body>
</html>

Questions:

1. Identify missing closing tags.
2. Analyze issues with the <img> tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

Answer:

Code:

```
<!DOCTYPE html>

<html>

<head>

    <title>Sample</title>

</head>

<body>

    <h1>Welcome</h1>

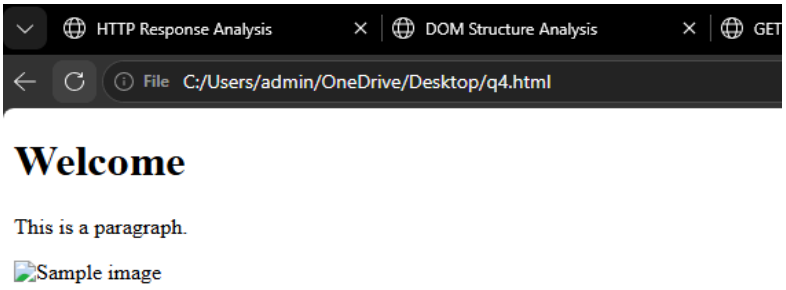
    <p>This is a paragraph.</p>

    <img src="image.jpg" alt="Sample image">

</body>

</html>
```

Output:



1. Identify missing closing tags.

The original code is missing </h1> and </p>.

2. Analyze the image tag.

The img element is missing the alt attribute. The img element is a void element and does not require a closing tag.

3. How will the browser handle the errors?

The browser attempts to correct missing closing tags and render the page. However, the output may not always match the intended structure.

4. Suggest a corrected version.

The corrected code includes the closing heading and paragraph tags and an alt attribute for the image.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Answer:

Code:

```
<!DOCTYPE html>

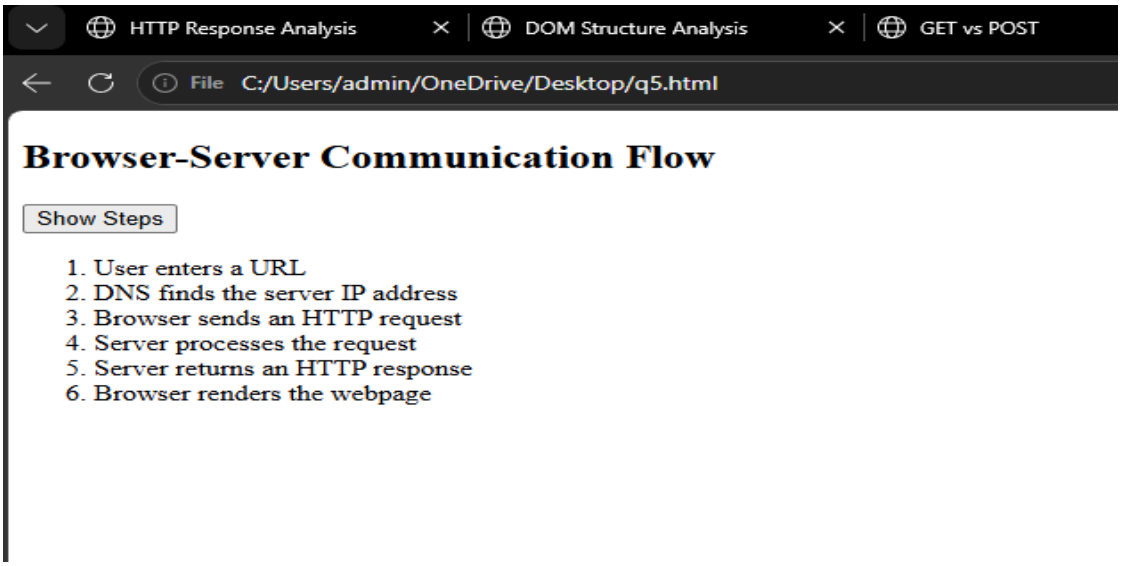
<html>

<head>

  <title>Browser-Server Communication</title>
```

```
</head>
<body>
    <h2>Browser-Server Communication Flow</h2>
    <button onclick="showSteps()">Show Steps</button>
    <ol id="steps"></ol>
    <script>
        function showSteps() {
            let steps = [
                "User enters a URL",
                "DNS finds the server IP address",
                "Browser sends an HTTP request",
                "Server processes the request",
                "Server returns an HTTP response",
                "Browser renders the webpage"
            ];
            document.getElementById("steps").innerHTML = "";
            steps.forEach(function(step) {
                let item = document.createElement("li");
                item.textContent = step;
                document.getElementById("steps").appendChild(item);
            });
        }
    </script>
</body>
</html>
```

Output:



1. Explain each step in the communication flow.

1. The user enters a URL in the browser.
2. DNS translates the domain name into an IP address.
3. The browser sends an HTTP request to the server.
4. The server processes the request.
5. The server returns an HTTP response.
6. The browser renders the webpage.

2. Identify where delays can occur.

Delays can occur during DNS lookup, connection establishment, request transmission, server processing, response transfer, and browser rendering.

3. Explain the role of DNS.

DNS stands for Domain Name System. It translates domain names into IP addresses so that the browser can locate the required server.

4. Suggest performance optimizations.

- Use DNS caching.
- Optimize images and compress files.
- Use a Content Delivery Network (CDN).
- Enable browser caching.
- Reduce unnecessary HTTP requests.
- Optimize server processing.